

WEEK – 4 (Skill)

Name: Anthony Nissy

Rollno: 2520030064

Section: 8

Experiment No. 7(Q7)

Title: Process Synchronization and Child Process Monitoring Using waitpid()

Aim: To implement process synchronization and child process monitoring using waitpid(), retrieve the PATH environment variable, parse search directories, locate executable files, verify permissions, handle missing commands, and test command resolution.

Objective:

Part 1 – Process Synchronization

- To create child processes.
- To synchronize parent and child processes using waitpid().
- To monitor child process execution.
- To obtain the child process exit status.
- To handle child process termination.
- To test process synchronization.

Part 2 – PATH-Based Command Resolution

- To retrieve the PATH environment variable.
- To parse directories stored in PATH.
- To search for executable files.
- To verify file permissions.
- To handle commands that cannot be found.
- To test command resolution.

Part 1: Process Synchronization and Child Process Monitoring

Theory:

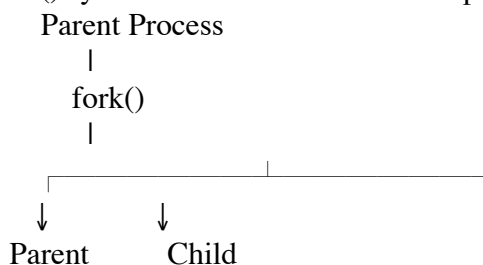
1. Process Synchronization

Process synchronization is the coordination of processes so that their execution occurs in the required order.

When a parent process creates a child process using fork(), both processes can execute independently. The parent can use waitpid() to wait for a particular child to finish.

2. fork()

The fork() system call creates a new child process.



The return value of fork() is:

- 0 → executing child process
- Positive value → executing parent process and contains child PID
- -1 → process creation failed

3. waitpid()

waitpid() allows the parent process to wait for a specific child process.

Syntax:

```
waitpid(pid, &status, 0);
```

where:

- pid → process ID of the child
- status → stores termination status
- 0 → parent waits until the child terminates

4. Child Process Monitoring

The parent can check whether the child terminated normally using:

```
WIFEXITED(status)
```

The exit status can then be obtained using:

```
WEXITSTATUS(status)
```

For example:

Child process completed.

Child exit status: 0

An exit status of 0 generally indicates successful execution.

5. Parent-Child Synchronization

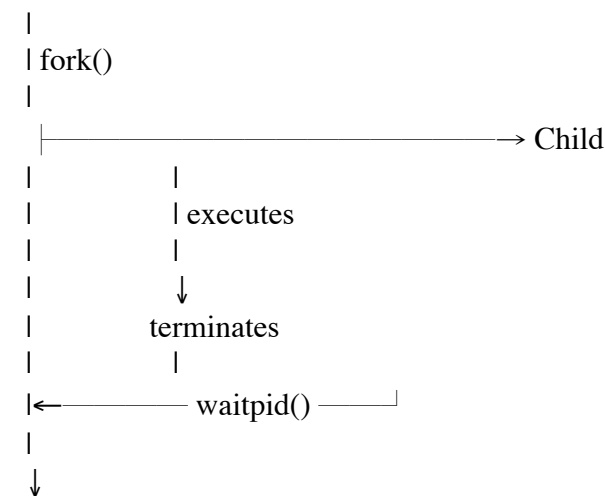
The parent waits for the child using:

```
waitpid(pid, &status, 0);
```

This prevents the parent from continuing before the child has finished.

The basic execution is:

Parent



Parent continues

Algorithm:

1. Start the program.
2. Create a child process using fork().
3. Check whether fork() was successful.
4. If the process is the child, perform the required task.

- ❑ Child prints its PID and performs its operation.
- ❑ Child terminates using `exit()`.
- ❑ If the process is the parent, obtain the child's PID.
- ❑ Call `waitpid()` to wait for the child.
- ❑ Check the child's termination status.
- ❑ Use `WIFEXITED()` to determine whether the child terminated normally.
- ❑ Use `WEXITSTATUS()` to obtain the exit status.
- ❑ Display the result.
- ❑ Stop.

Functions Used:

Function	Purpose
<code>main()</code>	Starting point of the program
<code>fork()</code>	Creates a child process
<code>waitpid()</code>	Waits for a specific child process
<code>getpid()</code>	Gets the current process ID
<code>getppid()</code>	Gets the parent process ID
<code>printf()</code>	Displays output
<code>sleep()</code>	Delays process execution
<code>exit()</code>	Terminates a process
<code>WIFEXITED()</code>	Checks normal child termination
<code>WEXITSTATUS()</code>	Gets child's exit status

Sample Output:

Parent process started.
Parent PID: 2501

Child process created.
Child PID: 2502
Parent PID: 2501

Child process completed.

Parent waiting for child...

Child terminated normally.
Child exit status: 0

Parent process completed.

Observation:

The child process is created successfully using `fork()`. The parent process waits for the child

using waitpid(). After the child terminates, the parent checks its termination status and retrieves its exit status using WIFEXITED() and WEXITSTATUS().

Result:

Process synchronization and child process monitoring were successfully implemented using fork() and waitpid(). The parent correctly waited for the child process and obtained its termination status.

Part 2 -PATH Variable Retrieval and Command Resolution

Aim-To retrieve the PATH environment variable, parse its search directories, locate executable files, verify execution permissions, handle missing commands, and test command resolution.

Theory:

1. PATH Environment Variable

PATH is an environment variable containing a list of directories where executable programs are located.

Example:

/usr/local/bin:/usr/bin:/bin

The directories are separated by a colon :.

When the user enters:

ls

the shell searches the directories in PATH to locate the executable.

2. Retrieving PATH

The getenv() function can be used to retrieve PATH.

Example:

```
char *path = getenv("PATH");
```

The returned value contains all directories in the search path.

3. Parsing PATH Directories

The directories are separated using :.

For example:

/usr/local/bin:/usr/bin:/bin

is divided into:

/usr/local/bin

/usr/bin

/bin

The program searches each directory one by one.

4. Locating Executables

Suppose the user enters:

ls

The program checks:

/usr/local/bin/ls

/usr/bin/ls

/bin/ls

until it finds the executable.

5. Verifying Permissions

Finding a file is not enough. The file must also have execute permission.

The `access()` function can be used:

`access(path, X_OK)`

`X_OK` checks whether the file can be executed.

6. Handling Missing Commands

If the command is not found in any directory in `PATH`, the program displays an error.

Example:

Enter command: `abcxyz`

Command not found: `abcxyz`

Algorithm:

- Start the program.
- Read the command name from the user.
- Retrieve the `PATH` environment variable using `getenv()`.
- Check whether `PATH` exists.
- Separate the `PATH` into individual directories using `:`.
- Take one directory at a time.
- Append the command name to the directory.
- Check whether the resulting file exists.
- Check whether the file has execute permission using `access()`.
- If an executable is found, display its complete path.
- Stop searching.
- If no executable is found, display Command not found.
- Stop.

Commands / Functions Used

1. gcc

Used to compile the C program.

`gcc path_resolver.c -o path_resolver`

2. ./path_resolver

Used to execute the compiled program.

`./path_resolver`

3. getenv()

Retrieves the value of an environment variable.

`getenv("PATH");`

4. strtok()

Splits the `PATH` string into individual directories using `:` as the delimiter.

`strtok(path, ":");`

5. access()

Checks whether a file exists and whether it has execute permission.

`access(fullpath, X_OK)`

6. snprintf()

Creates the complete path by combining the directory and command.

`snprintf(fullpath, sizeof(fullpath), "%s/%s", dir, command);`

Hardware and Operating System Relationship

Hardware and Operating System Relationship

Hardware refers to the physical components of the computer such as CPU, RAM, storage, keyboard, mouse and display.

The Operating System acts as an interface between application software and hardware. It manages hardware resources and provides services to programs.

Relationship:

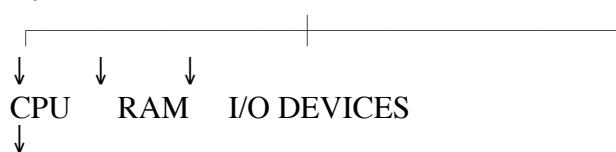
USER



APPLICATION SOFTWARE / SHELL



OPERATING SYSTEM



STORAGE DEVICES

How OS interacts with hardware:

- **CPU Management** – OS schedules processes and allocates CPU time.
- **Memory Management** – OS allocates and manages RAM.
- **File Management** – OS manages files stored on storage devices.
- **Device Management** – OS controls input/output devices.
- **Process Management** – OS creates, executes and terminates processes.
- **Security** – OS controls access to hardware and system resources.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>
#include <limits.h>

#define MAX_INPUT 100

int main()
{
    char command[MAX_INPUT];

    printf("Enter command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    if (strlen(command) == 0)
    {
```

```

    printf("Error: Empty command.\n");
    return 0;
}

/* Create child process */
pid_t pid = fork();

if (pid < 0)
{
    perror("fork failed");
    return 1;
}

/* Child process */
if (pid == 0)
{
    char *path;
    char *path_copy;
    char *directory;
    char fullpath[PATH_MAX];

    printf("\n--- Child Process ---\n");
    printf("Child PID: %d\n", getpid());
    printf("Parent PID: %d\n", getppid());

    /* Retrieve PATH */
    path = getenv("PATH");

    if (path == NULL)
    {
        printf("Error: PATH variable not found.\n");
        exit(1);
    }

    path_copy = strdup(path);

    if (path_copy == NULL)
    {
        perror("Memory allocation failed");
        exit(1);
    }

    /* Parse PATH directories */
    directory = strtok(path_copy, ":");

    while (directory != NULL)
    {
        snprintf(fullpath,
                 sizeof(fullpath),
                 "%s/%s",

```

```

        directory,
        command);

    /* Check executable permission */
    if (access(fullpath, X_OK) == 0)
    {
        printf("Command found: %s\n", fullpath);

        free(path_copy);
        exit(0);
    }

    directory = strtok(NULL, ":");
}

printf("Command not found: %s\n", command);

free(path_copy);
exit(1);
}

/* Parent process */
else
{
    int status;

    printf("\n--- Parent Process ---\n");
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    printf("Parent waiting for child...\n");

    if (waitpid(pid, &status, 0) == -1)
    {
        perror("waitpid failed");
        return 1;
    }

    printf("\n--- Child Monitoring ---\n");

    if (WIFEXITED(status))
    {
        printf("Child terminated normally.\n");
        printf("Child exit status: %d\n",
            WEXITSTATUS(status));
    }
    else
    {
        printf("Child did not terminate normally.\n");
    }
}

```



```
    printf("Parent process completed.\n");  
}  
  
return 0;  
}
```

OUTPUT: